

PyTorch Performance Analysis

from Profiling to Optimization

Dr. Cunliang (Liang) Geng

Senior Research Software Engineer

Email: c.geng@esciencecenter.nl

netherlands

eSciencecenter

UU INFOMQUDA Lecture 2026-10-05

Who we are & what we do

Netherlands eScience Center is the Dutch national centre of expertise for research software.

We work with researchers across disciplines to use **software, AI, data science, and advanced computing** to tackle challenging research questions.

Our **Research Software Engineers (RSEs)** work directly with research teams. They turn scientific ideas into working computational solutions.

We sit at the intersection of **research and software engineering** — using engineering skills not just to build software, but to **enable new science**.

All our projects are on <https://research-software-directory.org/>

<https://www.esciencecenter.nl/>



Upcoming opportunities

- Master student internships
- New RSE positions
- Research Software Engineering Doctorate program

Register our newsletter or follow our [LinkedIn](#) account to stay up to date

Newsletter <https://esciencecenter.us8.list-manage.com/subscribe?u=a0a563ca342f1949246a9f92f&id=31bfc2303d>

LinkedIn <https://www.linkedin.com/company/netherlands-escience-center/>



Hands-on Setup

On the server

1. SSH to the GPU server in a terminal on your laptop
2. ``git clone https://github.com/CunliangGeng/PyTorchPerformanceAnalysis.git`` to your server
3. Go to the cloned repo and run ``uv sync`` command

On your laptop

1. Install [VSCode](#) on your laptop (not the server), and install plugin "Python" and "Jupyter"
2. SSH to the server in VSCode, open the cloned repo in the workspace
3. Open the file "[notebooks/ep1.ipynb](#)" in VSCode, select the "[.venv/bin/python](#)" as the kernel, run the [SETUP cell](#) to check if everything works well



Hands-on notebooks

[notebooks/ep1.ipynb](#)

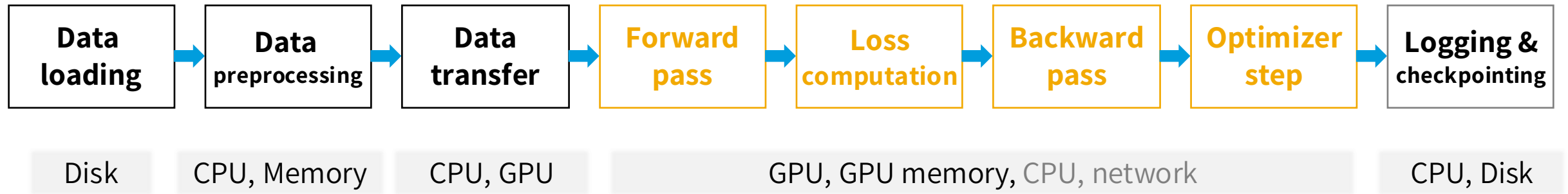
Timing one GPU operation correctly, and reading its profile as a timeline and as a table

[notebooks/ep2.ipynb](#)

Analyse a whole training loop: measure, profile, diagnose, optimise, verify, then next iteration



Training pipeline

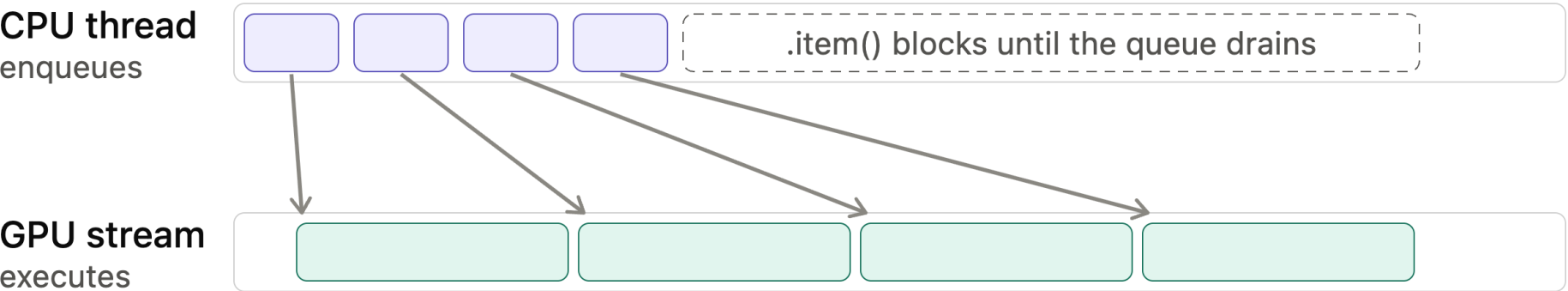


Training is a pipeline, not just “a model runs on GPU”.
If training is slow, any stage in the pipeline could be responsible

Training is a pipeline across *multiple hardware components*.
If training is slow, any hardware component could be responsible



GPU-bound: kernels queue behind the host



Arrows lean right as the queue grows; the next sync pays for all of it

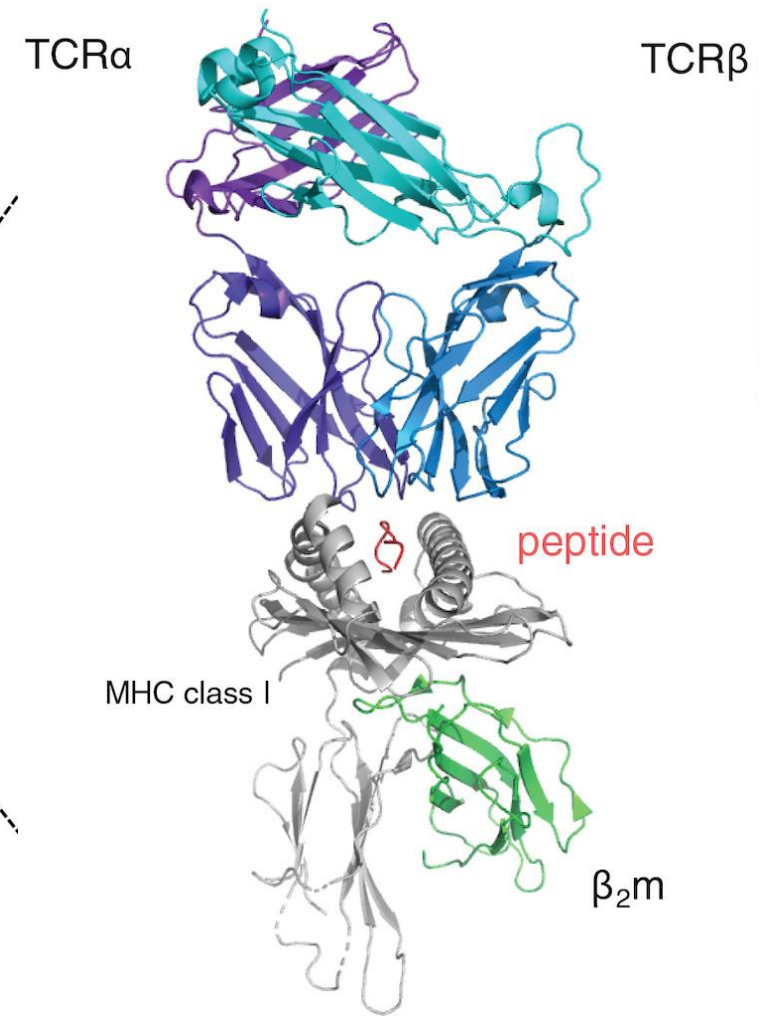
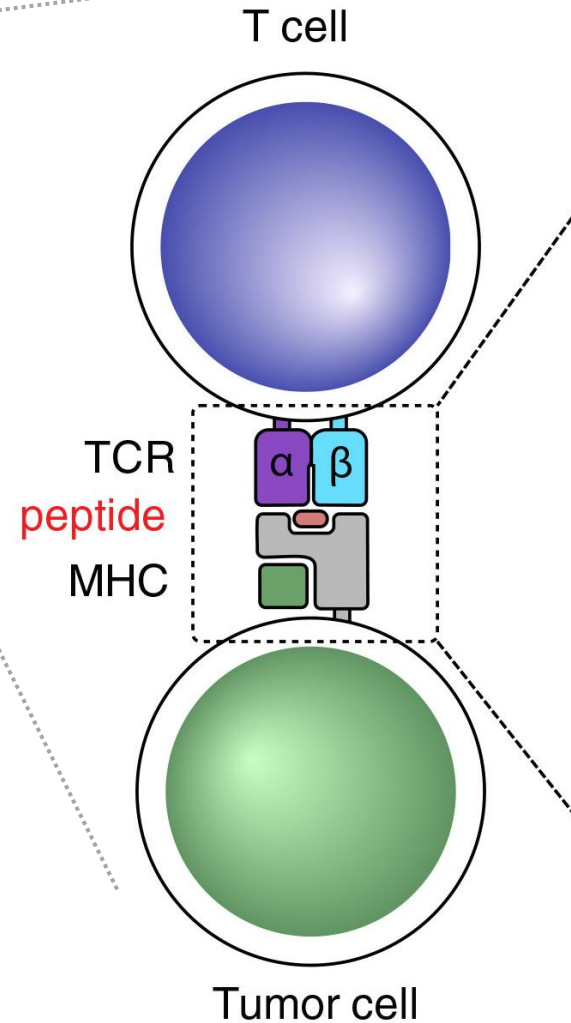
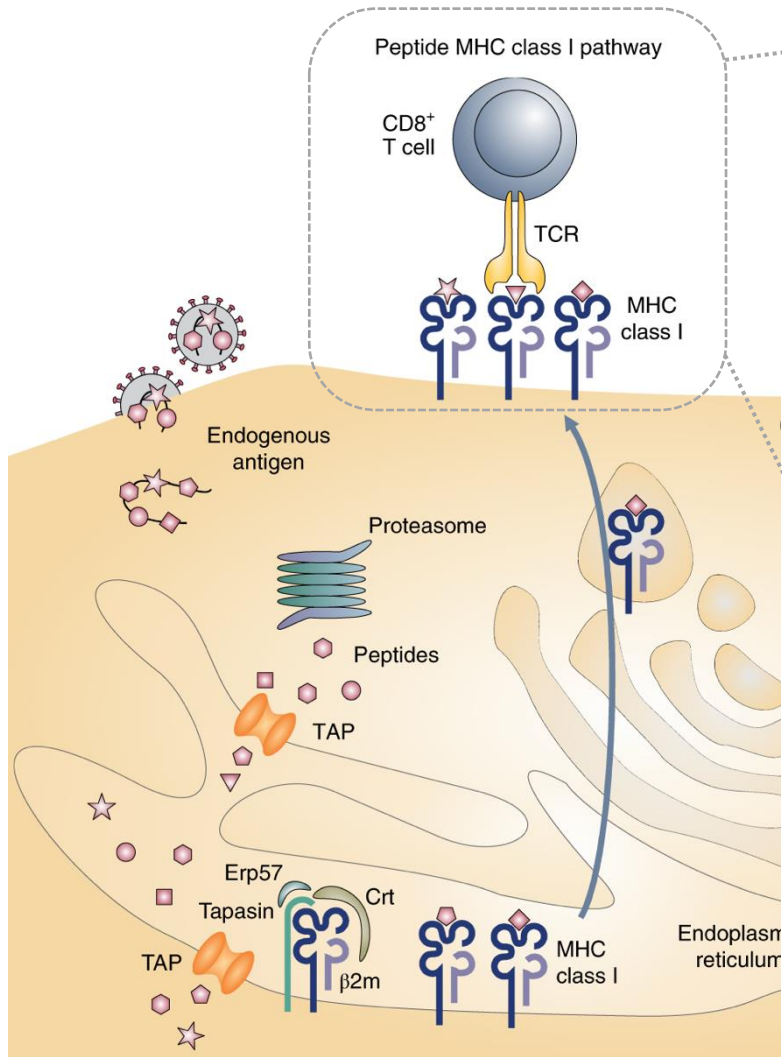
Host-bound: the GPU idles, waiting on the CPU



Near-vertical arrows and an idle GPU: faster kernels won't help here

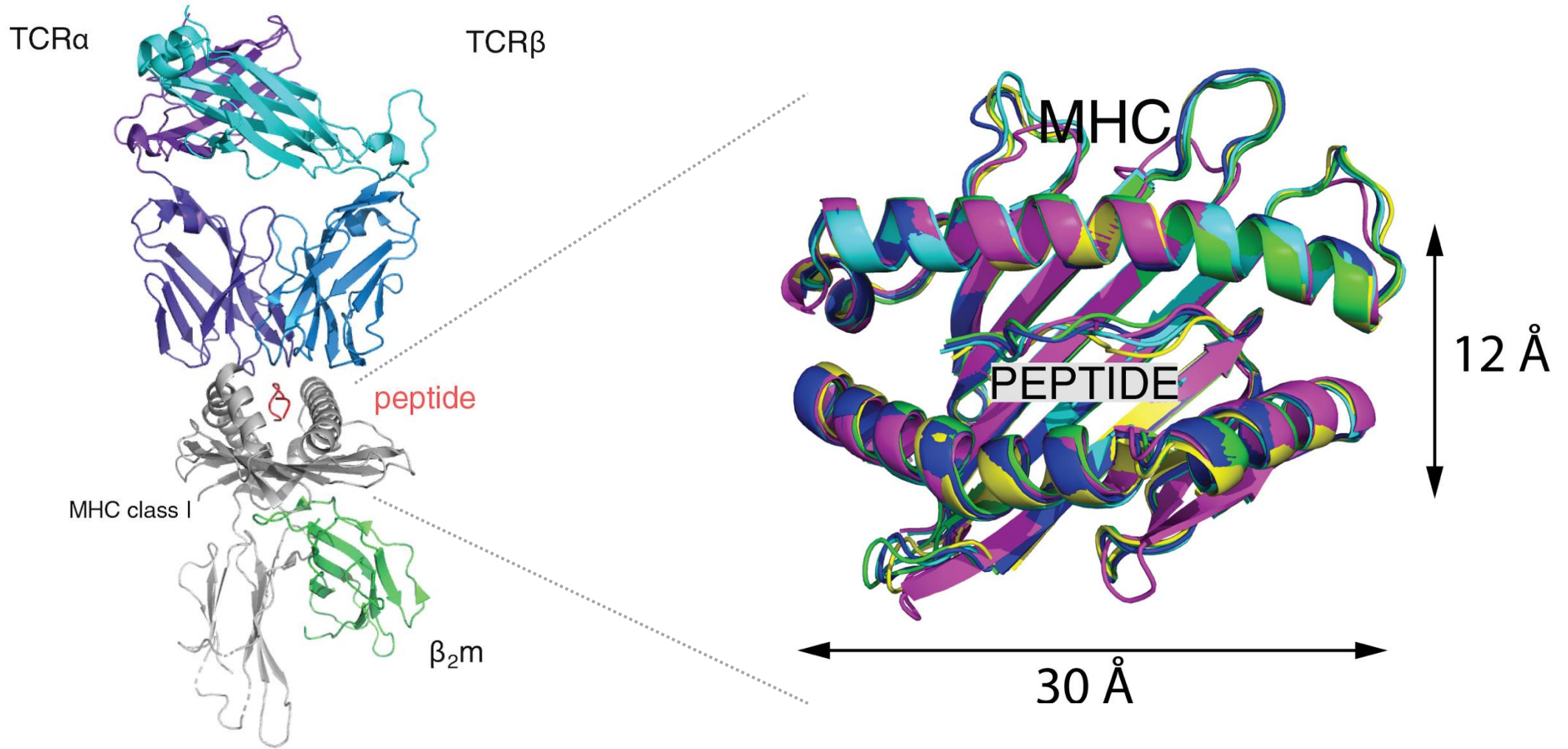


TCR-peptide-MHC complex critical to immune system



Major Histocompatibility Complex

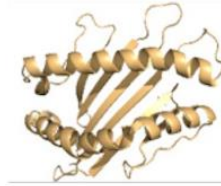
Predicting peptide strongly binding to MHC critical to vaccine design



SwiftMHC

HLA-A*02:01 allele

MHC structure

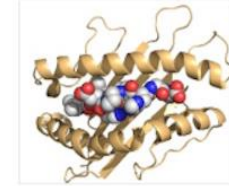


9-mer peptides

peptide sequence



all-atom structure



Binding affinity (BA)



SwiftMHC training time on a single A100 GPU

	batch size	8K data (1K epochs)	2M data (1K epochs)
		Training time in hours	Training time in days
phase 1 (baseline)	64	117	953
phase 1 (optimised)	512	3.5	3.9
phase 2 (baseline)	64	123	992
phase 2 (optimised)	512	3	16.3

Training on 2M dataset:
2000 days → 20 days

<https://doi.org/10.5281/zenodo.20272317>